SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1558

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Duration :60 Minutes

Code of Conduct

I **Dinesh V (192524309)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Dinesh V**

QUESTION 1: High-Volume Clickstream & Big Data Platform Reverse Engineering
A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Answer:

1. Distributed Storage System, Ingestion Framework, & Batch-Processing Tools

- **Ingestion Framework:** Apache Kafka / AWS Kinesis act as the central distributed message broker, ingestion layer capable of absorbing multi-gigabyte-per-second events with low latency. Fluentd or Logstash agents collect raw logs across servers and stream them directly into Kafka topics.
- Distributed Storage System: Data Lake Architecture built on Apache Hadoop HDFS or cloud object storage (AWS S3 / Google Cloud Storage / Azure Data Lake Storage Gen2). Raw data is organized into immutable landing zone folders partitioned by date (`year=YYYY/month=MM/day=DD`).
- Batch-Processing Tools: Apache Spark (running on AWS EMR or Databricks) executes scheduled nightly/hourly batch processing for heavy data transformations, aggregations, and historical modeling. Apache Airflow orchestrates dependencies and workflow schedules.

2. Structured vs. Semi-Structured Data Separation and Indexing

- **Data Separation:** Raw JSON clickstream payload (semi-structured) is passed into continuous ETL pipelines. Spark schema-on-read parses JSON payloads into tabular structured entities (transactions, user attributes) while preserving unstructured metadata in JSON/BSON columns.
- **Storage Format:** Structured data is stored in columnar columnar file formats like Apache Parquet or Apache ORC, providing high compression ratios (Snappy/Gzip) and column-pruning capabilities.
- **Indexing Strategy:** High-frequency point queries and full-text search use Elasticsearch / OpenSearch cluster indexes. Historical structured queries leverage hive-style partition keys and bloom filters inside Parquet files to eliminate unnecessary scan blocks.

3. Stream-Processing Engines for Real-Time Analytics

- **Engine Selection:** Apache Flink or Apache Spark Structured Streaming process events in real-time with sub-second latency.
- **Functionality:** Stateful stream processing maintains continuous tumbling/sliding time windows (e.g., 5-minute click counts, active session tracking). Flink's RocksDB state backend enables fault-tolerant, stateful event-driven computation.

4. Scalability and Partitioning Strategies

- **Horizontal Scaling:** Auto-scaling compute clusters (Kubernetes / EKS or Spark on YARN) dynamically expand executor pods based on CPU/Memory utilization and Kafka consumer group lag.

- **Data Partitioning:** Kafka topics use composite keys (e.g., `user_id` or `session_id`) to ensure sequential event ordering per user across topic partitions. Data lake files are partitioned by date and bucketed by `user_id` hash to prevent small file problems and optimize join operations.

5. Dashboard & Business Intelligence (BI) Connection Architecture

- **Serving Layer:** Processed batch data and real-time aggregations are loaded into an Analytical Data Warehouse (Snowflake, Databricks SQL, or AWS Redshift) and OLAP engines (Apache Pinot / ClickHouse).
- **BI Integration:** BI tools like Tableau, Power BI, or Looker connect using JDBC/ODBC connectors with direct query models for real-time dashboards and cached extracts for high-level executive summaries.

QUESTION 2: E-Commerce Event Streaming, Personalization & Recommendation Engine
An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Answer:

1. Event Streaming Tools, Customer Profiling Databases, & Recommendation Workflow

- **Event Streaming:** Apache Kafka or Redpanda capturing user events (`cart_add`, `checkout_abandon`, `product_view`) across multi-region deployment nodes.
- **Customer Profiling Databases:** Apache Cassandra or AWS DynamoDB stored as key-value/wide-column stores for low-latency (<10ms) reads/writes of real-time customer feature profiles.
- **Recommendation Workflow:** Two-stage recommendation architecture: (1) Retrieval stage using vector embeddings generated via Graph Neural Networks or ALS Matrix Factorization stored in a Vector DB (Milvus / Pinecone); (2) Ranking stage evaluating real-time context (current session clicks) using XGBoost or deep learning models.

2. Real-Time Synchronization of Session, Clickstream, & Transaction Data

- **Stream Join Architecture:** Apache Flink performs stateful stream-stream joins. Clickstream events are joined with session state using a shared `session_id`. Transaction completion events from payment gateways trigger a immediate state update in the online feature store.

3. Caching Systems, Distributed Query Engines, & ML Stages

- **Caching Systems:** Redis Enterprise cluster acting as an in-memory session cache and hot-feature store for instant personalized recommendations on product pages.
- **Distributed Query Engines:** Trino (formerly Presto) or StarRocks enabling ad-hoc queries across heterogeneous storage layers (joining transactional PostgreSQL with S3 Parquet logs).

- **ML Pipeline Stages:** Data preprocessing & Feature Store (Feast/Hopsworks) -> Offline Model Training (Spark MLlib / TensorFlow) -> Model Registry (MLflow) -> Real-time Inference Service (Triton / Seldon Core).

4. Merging Structured Order Data & Semi-Structured Browsing Data

- **Data Integration:** Semi-structured JSON clickstream logs are flattened in Delta Lake / Iceberg tables using Schema Evolution features. Structured relational order records (RDBMS) are ingested via Change Data Capture (Debezium/Kafka Connect) and merged into unified customer journey tables based on unified `user_id` keys.

5. Scalability & Fault-Tolerance Decisions

- **Scalability:** Global deployment using Amazon Route53 geo-routing. Edge event collection via CDN (Cloudflare/Fastly). Kafka consumer groups auto-scale with event volume increases.
- **Fault-Tolerance:** Multi-AZ and cross-region replication for DynamoDB/Cassandra. Kafka write replication factor = 3 (`min.insync.replicas=2`). Flink state checkpoints saved periodically to durable S3 storage.

6. Dashboards & Real-Time KPI Monitoring

- **Pipeline:** Flink streams metrics (Cart Abandonment Rate, GMV per minute, Regional Conversion Rate) into Apache Pinot/ClickHouse. Grafana and Superset pull from these OLAP layers to show real-time operational metrics, while Power BI displays daily strategic reports.

QUESTION 3: Big Data Healthcare Platform, Privacy & Clinical Analytics Pipeline
A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Answer:

1. Hybrid Database Architecture for Healthcare Information

- **Structured Data (EHR/Prescriptions):** Relational SQL (PostgreSQL / Google Cloud Healthcare API) enforcing HL7/FHIR standards for structured patient histories and billing.
- **Unstructured Data (Diagnostic Images):** Vendor Neutral Archives (VNA) built on AWS S3 / Blob Storage storing DICOM images, integrated with medical image indexing metadata in MongoDB.
- **Semi-Structured / Time-Series Data (Wearables):** TimescaleDB or InfluxDB dedicated to continuous high-frequency heart-rate, SPO2, and telemetry monitoring.

2. Interoperability & Data Integration Tools

- **Integration Middleware:** Apache NiFi and Mirth Connect (NextGen Connect) executing protocol transformations between legacy hospital HL7 v2 messages, lab LIMS formats, and FHIR RESTful APIs.
- **Insurance System Connectors:** EDI X12 processing engines converting 837/835 insurance claim formats into JSON/Parquet pipelines.

3. Streaming & Machine Learning Frameworks for Disease Prediction

- **Real-Time Streaming:** Apache Spark Streaming or Apache Flink processing real-time telemetry from wearable devices to alert nursing staff of acute distress signals.
- **ML/DL Frameworks:** PyTorch / TensorFlow Medical Imaging (MONAI framework) analyzing DICOM images for tumor/lesion detection. Random Forest / LightGBM models evaluating structured patient data for sepsis prediction and readmission risk.

4. Privacy, Encryption, & Compliance Framework (HIPAA/GDPR)

- **Data Encryption:** AES-256 encryption at rest; TLS 1.3 encryption in transit across all network endpoints.
- **Anonymization & De-identification:** Automated NLP pipelines (John Snow Labs NLP / AWS Comprehend Medical) stripping 18 Safe Harbor Protected Health Information (PHI) identifiers prior to analytical storage.
- **Access Control & Auditing:** Attribute-Based Access Control (ABAC) enforced via Apache Ranger / HashiCorp Vault. Immutable audit logs written to Write-Once-Read-Many (WORM) storage.

5. Clinical Decision Support & Population Health Analytics Pipeline

- **Analytics Architecture:** Medallion Architecture (Bronze -> Silver -> Gold). Raw FHIR/DICOM data (Bronze) -> Cleaned/De-identified EHR entities (Silver) -> Aggregated Risk Matrices & Disease Registries (Gold).
- **Clinical Decision Support (CDS):** Real-time alerts integrated directly into EHR systems via HL7 CDS Hooks. Population health dashboards powered by Databricks / Snowflake providing epidemiological trend analysis.

QUESTION 4: Smart City Edge Computing, IoT & Urban Analytics Platform
A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Answer:

1. Edge Computing Setup, Distributed Messaging, & Central Storage

- **Edge Computing Setup:** NVIDIA Jetson / Raspberry Pi gateway nodes installed at traffic intersections executing localized computer vision models to convert raw CCTV video into lightweight JSON metadata (car counts, pedestrian flow).
- **Distributed Messaging:** EMQX or Eclipse Mosquitto MQTT brokers at the edge layer, bridging data into central Apache Kafka clusters using MQTT-Kafka Connect.
- **Centralized Storage:** Data Lakehouse (Apache Iceberg on Ceph/S3) storing historical sensor telemetry and metadata logs.

2. Geospatial Analytics & Streaming Congestion Prediction

- **Geospatial Streaming:** Apache Flink integrated with GeoMeshing / H3 spatial indexing libraries. Vehicle telemetry streams are mapped into hexagonal geospatial grids (Uber H3).
- **Congestion Prediction ML:** Spatio-Temporal Graph Neural Networks (ST-GNN) deployed on stream processing engines predicting traffic bottlenecks 15–30 minutes in advance.

3. Time-Series & Location-Aware Databases

- **Time-Series DB:** TimescaleDB or ClickHouse for multi-billion record sensor metric storage.
- **Geospatial DB:** PostGIS (PostgreSQL extension) or MongoDB spatial indexes enabling complex bounding-box, radius, and route-intersection queries.

4. Lambda / Kappa Architecture for Combined Analytics

- **Real-Time Speed Layer:** Apache Flink computing live traffic density, re-routing public transit, and dispatching emergency response alerts.
- **Batch Layer:** Apache Spark aggregating multi-year traffic patterns, weather correlation, and carbon emission trends for long-term urban infrastructure planning.

5. Security & Access-Control Measures

- **Infrastructure Protection:** Mutual TLS (mTLS) authentication for edge device communication to prevent rogue node injection. Public key infrastructure (PKI) for IoT device identities.
- **Citizen Privacy:** CCTV feeds processed exclusively at the edge; zero video streams transmitted or saved to central servers. Face and license plate blurring applied immediately.

QUESTION 5: Integrated Healthcare Analytics, Real-Time Monitoring & Governance Framework
A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Answer:

1. Hybrid Storage Architecture for Sensitive Medical Data

- **Structured Clinical Data:** Managed Relational Database (AWS Aurora PostgreSQL / Azure SQL DB for Health) storing demographic, appointment, and lab records.
- **Unstructured Medical Records:** NoSQL Document DB (MongoDB/Couchbase) storing unstructured physician clinical notes, alongside Object Storage (S3) for PDF lab reports.
- **Wearable Sensor Streams:** Distributed Time-Series Store (InfluxDB / AWS Timestream) handling rapid IoT metric writes.

2. ETL Pipelines & Interoperability Standards

- **Interoperability Standards:** HL7 v2/v3, FHIR (Fast Healthcare Interoperability Resources) v4, and DICOM standards.
- **ETL Ingestion Pipelines:** Apache NiFi orchestrated with Airflow. Pipelines parse incoming FHIR bundle JSONs, harmonize disparate code sets (ICD-10, SNOMED CT, LOINC), and update central patient datamarts.

3. Real-Time Monitoring & Predictive Health Analytics

- **Telemetry Ingestion:** IoT Wearables connect via secure APIs -> AWS IoT Core -> Apache Kafka stream.
- **Predictive Engine:** Flink complex-event processing (CEP) rules evaluate physiological triggers (e.g., heart rate > 120 AND blood pressure drops). ML models score patient deterioration risk dynamically.

4. Distributed Processing in Population-Level Analysis

- **Framework:** Apache Spark / Delta Lake on Azure Databricks processing multi-terabyte datasets across millions of patients.
- **Functionality:** Stratifying population cohorts by comorbidities, evaluating regional disease prevalence, and measuring drug efficacy trends across demographic groups.

5. Encryption, Compliance, & Audit Governance Mechanisms

- **Compliance:** Strict adherence to HIPAA, HITECH, and GDPR regulations.

- **Security Controls:** Data encrypted with customer-managed keys (AWS KMS / Azure Key Vault). Field-level encryption (FLE) applied to Sensitive Personal Identifiable Information (PII). Role-Based (RBAC) and Attribute-Based Access Control (ABAC) managed via Immuta or Privacera.
- **Audit Logging:** Centralized, immutable audit trail captured via AWS CloudTrail and Splunk logging all data access events.

6. Machine Learning for Diagnosis Risk Scoring & Recommendations

- **Clinical Models:** Gradient Boosted Trees (CatBoost/LightGBM) trained on historical clinical features calculate Diagnosis Risk Scores (e.g., Readmission Risk, Diabetes Complication Index).
- **Treatment Recommendation:** Clinical decision trees and Transformer-based NLP models synthesize clinical guidelines and trial data to offer evidence-based clinical recommendations directly into the clinician interface.